



گزارش پروژه شماره ۲: الگوریتم‌های جستجوی محلی

لینک گیت‌هاب پروژه: <https://github.com/Elham-ch/AI-Projects>

۱ معرفی مسئله

در این پروژه، مسئله به صورت یک Grid World مدل شده است. در نقشه، خانه‌های هدف با T، مانع‌ها با X و خانه‌های آزاد با «.» مشخص می‌شوند. هدف این است که تعدادی حسگر در خانه‌های مجاز قرار دهیم، به طوری که تا حد ممکن همه هدف‌ها پوشش داده شوند. هر حسگر یک برد مشخص دارد و اگر فاصله منتهی آن تا یک هدف کمتر یا مساوی برد حسگر باشد، آن هدف پوشش داده می‌شود.

در فایل نقشه، دو عدد آخر به ترتیب برد حسگر و حداکثر تعداد حسگرهای مجاز هستند. برای نمونه در map1 برد حسگر برابر 2 و حداکثر تعداد حسگرها برابر 20 است. بنابراین الگوریتم اجازه دارد حداکثر ۲۰ حسگر را در خانه‌هایی قرار دهد که مانع نیستند. این مسئله از نوع بهینه‌سازی است، نه پیدا کردن یک مسیر از ابتدا تا هدف. یک جواب یا state در این پروژه لیستی از مختصات حسگرهاست، مثلاً:

```
1 [(10, 3), (7, 10), (1, 1)]
```

یعنی سه حسگر در سه مختصات مختلف قرار گرفته‌اند. چون ترتیب حسگرها اهمیتی ندارد، دو لیست زیر از نظر جواب مسئله یکسان هستند:

```
1 [(1, 1), (7, 10)]
2 [(7, 10), (1, 1)]
```

به همین دلیل در بخش‌هایی از کد، قبل از مقایسه حالت‌ها، لیست مختصات مرتب می‌شود.

۲ LocalSearchBase

بخش اصلی الگوریتم‌ها در کلاس LocalSearchBase قرار دارد. همه الگوریتم‌ها از این کلاس ارث‌بری می‌کنند، چون همه آن‌ها به چند کار مشترک احتیاج دارند: شناختن خانه‌های معتبر، محاسبه فاصله، تشخیص پوشش هدف‌ها، ساختن همسایه و ارزیابی کیفیت یک جواب. در کلاس، اطلاعات مورد نیاز از world گرفته می‌شود:

```
1 self.sensor_range = world.sensor_range
2 self.max_sensors = world.max_sensors
3 self.position_valid = world.is_valid_position
4 self.targets = world.get_targets()
```

به این ترتیب الگوریتم‌ها لازم نیست هر بار مستقیم با جزئیات محیط کار کنند. مثلاً وقتی لازم باشد بدانیم یک خانه مجاز است یا نه، کافی است از self.position_valid استفاده شود.

۱.۲ محاسبه فاصله و پوشش

فاصله بین حسگر و هدف با فاصله منتهی حساب می‌شود:

$$d((x_1, y_1), (x_2, y_2)) = |x_1 - x_2| + |y_1 - y_2|$$

در کد این فرمول در تابع distance آمده است:

```
1 return abs(a[0] - b[0]) + abs(a[1] - b[1])
```

بعد از آن، تابع covers بررسی می‌کند که آیا یک حسگر، یک خانه مشخص را پوشش می‌دهد یا نه:

```
1 return self.distance(sensor, position) <= self.sensor_range
```

بنابراین ناحیه پوشش هر حسگر به شکل یک لوزی منتهی است. تابع covered_targets هم روی همه هدف‌ها می‌چرخد و اگر حداقل یک حسگر آن هدف را پوشش دهد، هدف را داخل مجموعه هدف‌های پوشش داده‌شده قرار می‌دهد.

۲.۲ فرمول هزینه

تابع evaluate مهم‌ترین تابع مشترک پروژه است. همه الگوریتم‌ها با همین تابع کیفیت یک حالت را می‌سنجند. چون مسئله کمینه‌سازی است، هزینه کمتر بهتر است. فرمول هزینه به صورت زیر است:

$$Cost(s) = 200 \times U(s) + 1 \times O(s) + 10 \times |s|$$

که در آن:

- $U(s)$ تعداد هدف‌هایی است که هنوز پوشش داده نشده‌اند.
- $O(s)$ تعداد خانه‌هایی است که بیش از یک بار توسط حسگرها پوشش داده شده‌اند.
- $|s|$ تعداد حسگرهای استفاده‌شده در حالت فعلی است.

در کد، این بخش به شکل زیر نوشته شده است:

```
1 uncovered_target_cost = uncovered_count * 200
2 sensor_usage_cost = len(state) * 10
3 overlap_tile_cost = overlap_tile_count * 1
4
5 return uncovered_target_cost + overlap_tile_cost + sensor_usage_cost
```

دلیل انتخاب ضریب 200 برای هدف‌های پوشش داده‌نشده این است که پوشش هدف‌ها مهم‌ترین بخش مسئله است. اگر یک هدف پوشش داده نشود، جواب باید جریمه زیادی بگیرد تا الگوریتم ترجیح دهد حتی با اضافه کردن چند حسگر، آن هدف را پوشش دهد. ضریب 10 برای تعداد حسگرها باعث می‌شود وقتی دو جواب پوشش تقریباً مشابهی دارند، جواب با حسگر کمتر انتخاب شود. ضریب 1 برای همپوشانی کوچک‌تر است، چون همپوشانی بد است اما از پوشش ندادن هدف‌ها مهم‌تر نیست. در واقع اولویت‌ها این‌گونه تنظیم شده‌اند:

پوشش هدف‌ها ← کاهش تعداد حسگرها ← کاهش همپوشانی

تابع overlap_tile_count برای محاسبه همپوشانی، همه خانه‌های داخل برد هر حسگر را بررسی می‌کند. اگر خانه‌ای قبلاً توسط حسگر دیگری پوشش داده شده باشد، شمارنده همپوشانی زیاد می‌شود. این کار باعث می‌شود حسگرهایی که بیش از حد روی یک ناحیه مشترک افتاده‌اند جریمه شوند.

۳.۲ تولید همسایه‌ها

تابع generate_neighbors برای یک حالت فعلی، حالت‌های نزدیک به آن را تولید می‌کند. در جستجوی محلی، کیفیت همسایه‌ها خیلی مهم است، چون الگوریتم‌ها معمولاً با حرکت از یک حالت به همسایه‌های آن به جواب بهتر می‌رسند. در این پروژه سه نوع همسایه ساخته می‌شود:

- حذف یک حسگر از حالت فعلی.
- اضافه کردن یک حسگر جدید به یکی از موقعیت‌های کاندید.
- جابه‌جا کردن یک حسگر به یکی از چهار خانه بالا، پایین، چپ یا راست.

برای جلوگیری از تولید حالت تکراری، تابع داخلی add_neighbor هر کاندید را بعد از مرتب کردن به یک کلید تبدیل می‌کند:

```
1 key = tuple(sorted(candidate))
2 if key not in seen and len(candidate) <= self.max_sensors:
3     seen.add(key)
4     neighbors.append(candidate)
```

این کار مهم است، چون ترتیب حسگرها در لیست نباید باعث شود یک جواب چند بار به عنوان حالت متفاوت وارد جستجو شود. شرط $\text{len}(\text{candidate}) \leq \text{self.max_sensors}$ هم اجازه نمی‌دهد تعداد حسگرها از محدودیت نقشه بیشتر شود. برای اضافه کردن حسگر، کد فقط کاملاً تصادفی عمل نمی‌کند. ابتدا هدف‌هایی پیدا می‌شوند که هنوز پوشش داده نشده‌اند. سپس خانه‌های معتبر اطراف آن هدف‌ها به عنوان کاندید اضافه شدن حسگر در نظر گرفته می‌شوند. این انتخاب باعث می‌شود همسایه‌های تولیدشده بیشتر به سمت بهتر کردن پوشش هدف‌ها بروند. در کنار آن، تعدادی موقعیت تصادفی هم اضافه می‌شود تا الگوریتم‌ها کاملاً در یک ناحیه کوچک گیر نکنند:

```
1 random_candidates = random.sample(
2     self.valid_positions(),
3     k=min(len(self.valid_positions()), max(10, self.max_sensors * 2)),
4 )
```

عدد $\text{max}(10, \text{self.max_sensors} * 2)$ یعنی اگر تعداد حسگرهای مجاز کم باشد، حداقل ۱۰ کاندید تصادفی بررسی می‌شود و اگر تعداد حسگرهای مجاز زیاد باشد، حدود دو برابر آن موقعیت تصادفی وارد لیست کاندیدها می‌شود. این انتخاب تعادلی بین تنوع و زمان اجرا ایجاد می‌کند.

۳ الگوریتم Hill Climbing

الگوریتم Hill Climbing در فایل `hill_climbing.py` پیاده‌سازی شده است. ایده اصلی آن ساده است: از یک جواب شروع می‌کنیم، همه همسایه‌های آن را می‌سنجیم، بهترین همسایه را انتخاب می‌کنیم و اگر بهتر بود به آن حرکت می‌کنیم. این کار تا وقتی ادامه پیدا می‌کند که دیگر همسایه بهتری وجود نداشته باشد یا تعداد تکرارها به حد نهایی برسد. مقادیر پیش‌فرض الگوریتم در ابتدای تابع `run` مشخص شده‌اند:

```
1 max_iterations = kwargs.get("max_iterations", 500)
2 max_restarts = kwargs.get("max_restarts", 4)
3 sideways_limit = kwargs.get("sideways_limit", 8)
4 improvement_epsilon = kwargs.get("improvement_epsilon", 1e-9)
```

`max_iterations = 500` برای این انتخاب شده است که الگوریتم فرصت کافی برای اصلاح جواب داشته باشد، اما در نقشه‌های بزرگ بی‌دلیل طولانی اجرا نشود. `max_restarts = 4` به الگوریتم اجازه می‌دهد اگر در یک نقطه محلی گیر کرد، چند بار از یک حالت تصادفی جدید شروع کند. `sideways_limit = 8` برای حرکت‌های هم‌سطح است؛ یعنی وقتی همسایه هزینه برابر دارد، چند بار اجازه حرکت داده می‌شود تا شاید الگوریتم از یک ناحیه تخت عبور کند. `improvement_epsilon = 1e-9` برای مقایسه عددی استفاده شده است تا اختلاف‌های بسیار کوچک باعث تصمیم اشتباه نشوند. در شروع، اگر حالت اولیه خالی باشد، یک حالت تصادفی ساخته می‌شود:

```
1 current_state = [] if not initial_state else initial_state
2 if not current_state:
3     current_state = self.initialize_state()
```

سپس هزینه حالت فعلی محاسبه می‌شود و همان حالت به عنوان بهترین حالت اولیه ذخیره می‌شود. در هر تکرار، ابتدا همسایه‌ها ساخته می‌شوند:

```
1 neighbors = self.generate_neighbors(current_state)
```

اگر همسایه‌ای وجود نداشته باشد، جستجو متوقف می‌شود. در غیر این صورت، هزینه همه همسایه‌ها محاسبه می‌شود و کم‌هزینه‌ترین همسایه انتخاب می‌شود:

```
1 neighbor_costs = [
2     (self.evaluate(neighbor), neighbor) for neighbor in neighbors
3 ]
4 next_cost, next_state = min(neighbor_costs, key=lambda item: item[0])
```

بعد از انتخاب بهترین همسایه، سه حالت ممکن است پیش بیاید. اگر هزینه همسایه کمتر از هزینه فعلی باشد، الگوریتم به آن حرکت می‌کند و شمارنده حرکت‌های هم‌سطح صفر می‌شود. اگر هزینه برابر باشد و هنوز تعداد حرکت‌های هم‌سطح به حد ۸ نرسیده باشد، حرکت هم‌سطح انجام می‌شود. اگر نه بهبود وجود داشته باشد و نه حرکت هم‌سطح مجاز باشد، الگوریتم تا ۴ بار اجازه شروع دوباره دارد:

```
1 elif restarts_used < max_restarts:
2     current_state = self.initialize_state()
3     current_cost = self.evaluate(current_state)
4     restarts_used += 1
5     sideways_used = 0
```

این طراحی باعث می‌شود Hill Climbing از حالت کاملاً حریصانه کمی بهتر شود. نسخه حریصانه ممکن است خیلی زود در یک بهینه محلی متوقف شود، اما در این پیاده‌سازی حرکت هم‌سطح و شروع دوباره شانس خروج از چنین نقاطی را افزایش می‌دهد. با این حال، الگوریتم هنوز تضمین نمی‌کند جواب سراسری پیدا شود، چون فقط همسایه‌های نزدیک حالت فعلی را می‌بیند.

۴ الگوریتم Simulated Annealing

الگوریتم Simulated Annealing در فایل simulated_annealing.py پیاده‌سازی شده است. این الگوریتم شبیه Hill Climbing با همسایه‌ها کار می‌کند، اما یک تفاوت مهم دارد: گاهی به جواب بدتر هم حرکت می‌کند. دلیل این کار فرار از بهینه‌های محلی است. در ابتدای اجرا دما زیاد است و الگوریتم راحت‌تر حرکت بدتر را قبول می‌کند. با گذشت زمان، دما کم می‌شود و الگوریتم محافظه‌کارتر عمل می‌کند. تابع زمان‌بندی دما به این صورت است:

$$schedule(t) = t_0 \times \alpha^t$$

در کد:

```
1 def schedule(self, initial_temperature, cooling_rate, t):
2     return initial_temperature * math.pow(cooling_rate, t)
```

مقادیر پیش‌فرض الگوریتم عبارت‌اند از:

```
1 max_iterations = kwargs.get("max_iterations", 1000)
2 initial_temperature = kwargs.get("initial_temperature", 100.0)
3 cooling_rate = kwargs.get("cooling_rate", 0.995)
4 min_temperature = kwargs.get("min_temperature", 0.001)
```

دمای اولیه 100 نسبت به مقیاس تابع هزینه مقدار متوسط و قابل قبولی است. چون جریمه یک هدف پوشش داده‌نشده 200 است، دمای 100 اجازه می‌دهد در ابتدای کار بعضی حرکت‌های بدتر پذیرفته شوند، اما حرکت‌های خیلی بد همچنان احتمال کمی دارند. نرخ سرد شدن 0.995 باعث کاهش آرام دما می‌شود. اگر نرخ خیلی کوچک باشد، دما سریع کم می‌شود و الگوریتم شبیه Hill Climbing می‌شود. اگر خیلی نزدیک به ۱ باشد، زمان زیادی صرف جستجوی تصادفی می‌شود. مقدار 1000 برای max_iterations نیز به عنوان محدودیت عملی زمان اجرا در نظر گرفته شده است. چون این الگوریتم به صورت تصادفی همسایه انتخاب می‌کند و ممکن است حرکت‌های بدتر را هم بپذیرد، اگر هیچ سقفی برای تعداد تکرارها نداشته باشیم، زمان اجرا مخصوصاً در نقشه‌های بزرگ قابل پیش‌بینی نیست. با این نرخ سرد شدن، دما در پایان ۱۰۰۰ تکرار هنوز حدود 0.67 است، پس در اجرای پیش‌فرض معمولاً شرط تعداد تکرار باعث توقف می‌شود، نه شرط min_temperature. در هر تکرار، یک همسایه تصادفی از حالت فعلی گرفته می‌شود:

```
1 next_state = self.get_random_neighbor(current_state)
2 next_cost = self.evaluate(next_state)
```

در این پیاده‌سازی مقدار delta به صورت اختلاف هزینه حالت فعلی و همسایه محاسبه می‌شود. بنابراین اگر delta مثبت باشد، همسایه هزینه کمتری دارد و همیشه پذیرفته می‌شود:

```
1 delta = current_cost - next_cost
2 if delta > 0:
3     current_state = next_state
4     current_cost = next_cost
```

اگر همسایه بدتر باشد، با احتمال زیر پذیرفته می‌شود:

$$P = e^{\Delta/T}$$

که در آن Δ برابر current_cost - next_cost است. برای حرکت بدتر، این مقدار منفی می‌شود؛ پس اگر همسایه خیلی بدتر باشد یا دما کم باشد، مقدار $e^{\Delta/T}$ کوچک‌تر شده و احتمال پذیرش کاهش پیدا می‌کند:

```
1 accept_probability = math.exp(delta / temperature)
2 if random.random() < accept_probability:
3     current_state = next_state
4     current_cost = next_cost
```

حتی اگر الگوریتم به یک حالت بدتر حرکت کند، بهترین حالت دیده‌شده تا آن لحظه جداگانه ذخیره می‌شود. بنابراین خروجی نهایی بهترین حالتی است که در کل مسیر دیده شده، نه لزوماً آخرین حالت.

۵ الگوریتم ژنتیک

الگوریتم ژنتیک در فایل genetic_algorithm.py پیاده‌سازی شده است. در این روش به جای اینکه فقط یک جواب را تغییر دهیم، یک جمعیت از جواب‌ها نگه می‌داریم. در هر نسل، افراد بهتر شانس بیشتری برای انتخاب شدن به عنوان والد دارند. سپس از ترکیب والد‌ها فرزند ساخته می‌شود و گاهی روی فرزند جهش انجام می‌شود. مقادیر پیش‌فرض الگوریتم در ابتدای تابع run آمده‌اند:

```
1 generations = kwargs.get("generations", kwargs.get("max_iterations", 1000))
2 population_size = max(2, kwargs.get("population_size", 30))
3 mutation_rate = kwargs.get("mutation_rate", 0.1)
4 fit_enough = kwargs.get("fit_enough", 0)
```

تعداد نسل‌ها 1000 است تا الگوریتم فرصت کافی برای ترکیب جواب‌ها داشته باشد. اندازه جمعیت 30 انتخاب شده است، چون اگر جمعیت خیلی کوچک باشد تنوع کم می‌شود و اگر خیلی بزرگ باشد هر نسل زمان بیشتری می‌گیرد. نرخ جهش 0.1 یعنی حدود ۱۰ درصد فرزندان تغییر تصادفی پیدا می‌کنند. این مقدار برای حفظ تنوع مناسب است، اما آنقدر زیاد نیست که رفتار الگوریتم کاملاً تصادفی شود. $fit_enough = 0$ یعنی اگر هزینه به صفر برسد، جواب کامل پیدا شده و اجرا متوقف می‌شود. البته با توجه به جریمه تعداد حسگرها، هزینه صفر فقط زمانی ممکن است که هیچ حسگری لازم نباشد یا تابع هزینه در یک نقشه خاص اجازه چنین حالتی بدهد؛ در بیشتر نقشه‌ها الگوریتم با حد نسل متوقف می‌شود.

۱.۵ جمعیت اولیه و تابع fitness

جمعیت اولیه با خود حالت اولیه شروع می‌شود و بقیه افراد به صورت تصادفی ساخته می‌شوند:

```
1 population = [initial_state]
2
3 while len(population) < population_size:
4     population.append(self.random_individual())
```

هر فرد یک لیست از مختصات حسگرهاست. تابع random_individual ابتدا همه موقعیت‌های معتبر را می‌گیرد، سپس تعداد حسگرها را به طور تصادفی بین ۱ و بیشینه مجاز انتخاب می‌کند:

```
1 sensor_count = random.randint(1, max_count)
2 return random.sample(valid_positions, sensor_count)
```

استفاده از random.sample باعث می‌شود یک موقعیت دوبار در یک فرد تکرار نشود. چون تابع اصلی پروژه هزینه را کمینه می‌کند، برای انتخاب ژنتیکی لازم است آن را به شایستگی تبدیل کنیم، یعنی مقدار بزرگ‌تر بهتر باشد. این کار با فرمول زیر انجام شده است:

$$fitness(s) = \frac{1}{1 + Cost(s)}$$

در کد:

```
1 def fitness(self, individual):
2     return 1 / (1 + self.evaluate(individual))
```

اضافه شدن عدد ۱ در مخارج برای جلوگیری از تقسیم بر صفر است. هرچه هزینه کمتر باشد، مقدار شایستگی بزرگ‌تر می‌شود و فرد شانس بیشتری برای انتخاب به عنوان والد دارد.

۲.۵ انتخاب، تولیدمثل و جهش

در هر نسل، ابتدا وزن انتخاب هر فرد بر اساس شایستگی آن محاسبه می‌شود:

```
1 weights = self.weighted_by(population)
```

سپس برای تولید هر فرزند، دو والد با random.choices و با توجه به همین وزن‌ها انتخاب می‌شوند:

```
1 parent1, parent2 = random.choices(
2     population,
3     weights=weights,
4     k=2
5 )
```

تابع reproduce از برش یک نقطه‌ای استفاده می‌کند. ابتدا یک نقطه c بین ۱ و طول والد اول انتخاب می‌شود. بخش اول فرزند از والد اول و بخش دوم آن از والد دوم گرفته می‌شود:

```
1 c = random.randint(1, n)
2 child = parent1[:c] + parent2[c:]
```

بعد از تولید فرزند، تابع clean_individual اجرا می‌شود. این تابع موقعیت‌های تکراری یا نامعتبر را حذف می‌کند و اگر تعداد حسگرها از حد مجاز بیشتر شود، بقیه را کنار می‌گذارد. این مرحله لازم است، چون ترکیب دو والد ممکن است فرزندی بسازد که در آن یک مختصات چند بار آمده باشد. جهش با احتمال 0.1 روی فرزند اعمال می‌شود:

```
1 if random.random() < mutation_rate:
2     child = self.mutate(child)
```

در تابع mutate یکی از سه عمل add، remove یا replace تصادفی انتخاب می‌شود. در add یک حسگر جدید به یک موقعیت معتبر اضافه می‌شود، در remove یکی از حسگرها حذف می‌شود، و در replace یکی از حسگرها با یک موقعیت معتبر دیگر عوض می‌شود. اگر یک عمل به خاطر محدودیت‌ها ممکن نباشد، بخش else در عمل بیشتر نقش جایگزینی را دارد. در این پیاده‌سازی، بهترین فرد هر نسل برای ساخت نسل بعدی به صورت مستقیم نگه داشته نمی‌شود، یعنی elitism صریح نداریم. با این حال، بهترین جواب دیده‌شده در کل اجرا در متغیرهای best_state و best_cost ذخیره می‌شود. بنابراین حتی اگر بهترین فرد از جمعیت حذف شود، جواب نهایی از بین نمی‌رود.

۶ الگوریتم Beam

Beam Search در فایل beam_search.py پیاده‌سازی شده است. این الگوریتم از نظر ایده بین جستجوی حریصانه و جستجوی کامل قرار می‌گیرد. به جای اینکه فقط یک حالت فعلی داشته باشیم، در هر مرحله چند حالت خوب نگه می‌داریم. تعداد این حالت‌ها با beam_width مشخص می‌شود. در شروع تابع run دو مقدار پیش‌فرض داریم:

```
1 max_iterations = kwargs.get("max_iterations", 100)
2 beam_width = max(1, kwargs.get("beam_width", 5))
```

beam_width = 5 یعنی در هر مرحله فقط ۵ حالت برتر نگه داشته می‌شود. اگر این عدد ۱ باشد، الگوریتم تقریباً شبیه جستجوی حریصانه می‌شود. اگر خیلی بزرگ باشد، زمان اجرا و تعداد ارزیابی‌ها زیاد می‌شود. عدد ۵ برای این پروژه تعادل مناسبی ایجاد می‌کند. max_iterations = 100 نیز به عنوان سقف عملی انتخاب شده است، چون در هر تکرار همه همسایه‌های همه اعضا ساخته و ارزیابی می‌شوند. تابع initial_beam مکان اولیه را می‌سازد. اگر حالت اولیه وجود داشته باشد، اول همان وارد می‌شود. سپس تا وقتی تعداد حالت‌ها به beam_width برسد، حالت‌های تصادفی غیرتکراری ساخته می‌شوند:

```
1 while len(beam) < beam_width and attempts < beam_width * 10:
2     state = self.random_state()
3
4     if not self.seen_before(state, beam):
5         beam.append(state)
6
7     attempts += 1
```

شرط attempts < beam_width * 10 برای جلوگیری از حلقه بی‌نهایت است. چون حالت‌ها تصادفی ساخته می‌شوند، ممکن است چند بار حالت تکراری تولید شود. این شرط به کد اجازه می‌دهد برای ساخت هر عضو حدود ۱۰ بار تلاش کند، اما اگر حالت جدید پیدا نشد، برنامه گیر نکند. در حلقه اصلی، ابتدا برای همه حالت‌ها همسایه تولید می‌شود:

```
1 for state in beam:
2     neighbors = self.generate_neighbors(state)
3
4     if not neighbors:
5         successors.append(state)
6     else:
7         successors.extend(neighbors)
```

اگر یک حالت همسایه نداشته باشد، خود آن حالت داخل successors نگه داشته می‌شود. این کار باعث می‌شود یک جواب خوب فقط به خاطر نداشتن همسایه حذف نشود. اگر همسایه داشته باشد، همه همسایه‌ها به لیست جانشین‌ها اضافه می‌شوند. بعد از جمع شدن همه جانشین‌ها، اگر این لیست خالی باشد جستجو متوقف می‌شود.

سپس تابع `best_states` اجرا می‌شود. این تابع اول حالت‌های تکراری را حذف می‌کند، بعد برای هر حالت هزینه را محاسبه می‌کند، حالت‌ها را بر اساس هزینه صعودی مرتب می‌کند و فقط `beam_width` حالت اول را برمی‌گرداند:

```
1 scored_states = [(self.evaluate(state), state) for state in unique_states]
2 scored_states.sort(key=lambda item: item[0])
3
4 return [state for _, state in scored_states[:beam_width]]
```

در پایان هر تکرار، بهترین حالت با بهترین حالت کلی مقایسه می‌شود. اگر هزینه آن کمتر باشد، جواب نهایی به‌روزرسانی می‌شود. اگر هزینه به صفر برسد، الگوریتم زودتر متوقف می‌شود:

```
1 if best_cost == 0:
2     break
```

از آنجا که تابع هزینه شامل جریمه تعداد حسگرهاست، رسیدن به صفر در نقشه‌های معمولی خیلی محتمل نیست، اما این شرط برای کامل بودن الگوریتم و توقف سریع در حالت ایده‌آل گذاشته شده است.

۷ مقایسه رفتار الگوریتم‌ها

هر چهار الگوریتم از یک تابع هزینه و یک تابع تولید همسایه استفاده می‌کنند، اما روش حرکت آن‌ها در فضای جواب متفاوت است.

- Hill Climbing: سریع و ساده است، اما ممکن است در بهینه محلی گیر کند. حرکت هم‌سطح و شروع دوباره این مشکل را کمتر می‌کند.
- Simulated Annealing: نسبت به صعود تپه‌ای انعطاف بیشتری دارد، چون در دمای بالا گاهی حرکت بدتر را می‌پذیرد. کیفیت آن به برنامه سرد شدن وابسته است.
- Genetic Algorithm: چند جواب را هم‌زمان بررسی می‌کند و با ترکیب والد‌ها می‌تواند نواحی مختلف فضای جستجو را امتحان کند. در عوض، به جمعیت، جهش و تعداد نسل‌ها حساس است.
- Beam Search: در هر مرحله چند جواب خوب را نگه می‌دارد و از حذف زودهنگام همه مسیرها جلوگیری می‌کند. اگر عرض پرتو کوچک باشد تنوع کم می‌شود و اگر بزرگ باشد هزینه محاسباتی بالا می‌رود.

برای داشتن یک مقایسه عددی، همه الگوریتم‌ها روی نقشه‌های `map1` تا `map7` اجرا شدند. قبل از اجرای هر الگوریتم مقدار `random.seed(42)` تنظیم شد تا نتیجه‌ها قابل تکرار باشند. جدول زیر هزینه نهایی هر الگوریتم را نشان می‌دهد.

Beam Search	Genetic Algorithm	Simulated Annealing	Hill Climbing	نقشه
130	146	175	130	map1
1480	1480	1880	1480	map2
653	671	452	655	map3
440	440	440	440	map4
350	2975	360	350	map5
3460	3266	3870	3062	map6
70	132	196	60	map7

در این اجرای نمونه، صعود تپه‌ای در بیشتر نقشه‌ها کمترین هزینه یا یکی از کمترین هزینه‌ها را به دست آورد. Simulated Annealing در `map3` بهترین نتیجه را داشت، چون پذیرش حرکت‌های بدتر به آن کمک کرد از مسیرهای حریصانه فاصله بگیرد. Beam Search در `map1` و `map5` با صعود تپه‌ای برابر شد و در `map7` به جواب نزدیک به آن رسید. الگوریتم ژنتیک با وجود افزایش حد نسل به 1000 در بعضی نقشه‌ها هنوز نتیجه ضعیف‌تری داشت، که می‌تواند به نبود elitism صریح و ساده بودن عملگرهای انتخاب، ترکیب و جهش مربوط باشد. البته چون الگوریتم‌ها تصادفی هستند، برای نتیجه‌گیری دقیق‌تر بهتر است هر الگوریتم چند بار اجرا و میانگین هزینه گزارش شود.